

✓ 1. Literature & Open-Source Code Survey

a. N. Sudhakar Yadav, R. Aluvalu, U. M. Viswanadhula, M. V. V. P. Kantipudi, P. K. N. Shankar Murthy, and S. Salendra, "Quantum Leap in Automation: Exploring Quantum Machine Learning for Enhanced Precision in Optoelectronic Robotic Systems," International Journal of Energy and Applications, 2023. [Online]. Available:

<https://iijeta.org/download/file/fid/176175>

- Loaded/Clean Dataset: They did not describe in detail how they loaded the dataset, they didn't mention paths, CSV handling or libraries. The authors used the Kaggle dataset for evaluation and illustrate data flow their AQE-DF framework(pg1172). They did not explain any missing value handling or traditional data cleaning steps.
- Visualization/EDA: They didn't perform traditional exploratory data analysis(EDA), their focus was on designing a quantum-enhanced decision framework(AQE-DF). They had quantum decision-making module, a quantum architecture diagram, a feature extraction unit, and a quantum feature transformation unit. Their analysis came from how the data flows through their quantum system.
- Preprocessing Steps: they do not explain traditional preprocessing, instead they focus on quantum feature extraction, quantum feature transformation, quantum entanglement-based decision fusion. Their preprocessing happens inside the quantum architecture, not traditional ML pipeline. So they discuss feature encoding and transformation within a quantum processing framework, not classical ML.

b. S. Pookkuttath, M. R. Elara, V. Sivanantham, and B. Ramalingam, "AI-enabled predictive maintenance framework for autonomous mobile cleaning robots," Sensors, vol. 22, no. 1, p. 13, Dec. 2021, doi: 10.3390/s22010013.

- Loaded/cleaned Dataset: The authors collected the vibration signal data using IMU sensors mounted on autonomous mobile cleaning robots, and organized the dataset into 5 labeled vibration classes. The dataset

was split into training, validation(5-fold cv), and sperate test sets(500 samples per class)

- EDA/Visualization: The authors have time-amplitude plots of raw vibration signals for all 5 classes across different signal types and axes. The graphs show how vibration patterns differ between operational conditions before model training.
- Preprocessing Steps: emphasized signal normalization to a -1 to +1 range during preprocessing. Dimensionality reduction is done through pooling layers within CNN architecture and dropout regularization to reduce overfitting. Model performance evaluation based on multiclass accuracy, precision, recall, and F1 score.

c. D. K. Yadav, A. Kaushik, and N. Yadav, "Predicting machine failures using machine learning and deep learning algorithms," Sustain. Manuf. Serv. Econ., vol. 3, p. 100029, Jul. 2024, doi: 10.1016/j.smse.2024.100029.

- Loaded/Cleaned Dataset: Authors used a publicly available predictive maintenance dataset from UCI Machine Learning Repository, that contains operational machine parameters and failure labels. The dataset was highly imbalanced so SMOTE was applied to balance the classes before model development. The dataset was prepared for supervised learning by putting features into labeled failure categories and splitting into training and testing 4:1 ratio.
- Visualization/EDA: A correlation heatmap was used to examine relationships between features and guide feature selection. VIF(variance inflation analysis) was performed to assess multicollinearity among the independent variables.
- Preprocessing steps: Smote to correct class imbalance, minmax normalization to scale features, feature selection based on correlation heatmap analysis, removal of highly correlated variables, removing redudent columns, removal of noninformative features. The final training dataset was reduced from 13 features to 5 selected before modeling.

d. Mohammed, N. H. (2023). Machine Learning–Based Predictive Maintenance in Industrial Robotics. AIJES.

- Loaded/Clean Dataset: The paper discusses robotic sensor data collection and Industrial IoT integration. It emphasizes structured data handling but does not describe detailed CSV loading or explicit missing value

handling steps.

- Visualization/EDA: No traditional EDA plots are shown. The focus is on conceptual data flow and system architecture rather than statistical exploration.
- Preprocessing Steps: The study highlights handling noisy sensor signals, data consistency, and feature preparation before applying supervised or deep learning models.

e. Pinto, R., & Cerquitelli, T. (2019). Robot fault detection and remaining life estimation for predictive maintenance. *Procedia Computer Science*.

- Loaded/Clean Dataset: Real operational robotic data was collected and organized into structured datasets before training. The dataset was split into training and validation sets.
- Visualization/EDA: Time-series signal visualization was performed to analyze vibration patterns before modeling.
- Preprocessing Steps: Sensor data was transformed into meaningful features. Feature selection and normalization were applied before model implementation. Survival analysis and ML models were used for fault detection and RUL estimation.

f. F. E. Bezerra et al., "Impacts of Feature Selection on Predicting Machine Failures Based on Synthetic Industrial Data," *Applied Sciences*, vol. 14, no. 8, p. 3337, 2024.

- Loaded/Clean Dataset: The authors collected predictive maintenance dataset from UC Irving Machine Learning Repository containing 10,000 records with attributes including UID, ProductID, L/M/H categories, air temperature, process temperature, rotation speed, torque, tool wear, target label (Machine failure yes/no) and failure type. They state they had an imbalanced dataset, much higher no failure samples compared to individual failure.
- Visualization/EDA: they describe correlation-based techniques such as PCA variance analysis, mRMR, NCA and

Random feature importance to evaluate relationships between variables and determine feature relevance.

- Preprocessing Steps: Dataset was normalized to interval [0,1] using MinMaxScaler from scikit learn to address differences in variable scales. Data was split into 70/30% training and testing. Feature selection and dimensionality reduction technique was applied, which included: PCA, mRMR, NCA, DAE, and Random Forest-based feature importance analysis. Random Forest was then used for failure classification, evaluated with accuracy, precision, and recall.

✓ Open-Source Code

Source: Baressiegota, S. (n.d.). Example of Data Loading for Robot Kinematics Datas. Kaggle.

<https://www.kaggle.com/code/sandibaressiegota/example-of-data-loading-for-robot-kinematics-datas>

In this Kaggle notebook, the dataset is loaded using the pandas library. The file is accessed from Kaggle's input directory using a relative path: `df = pd.read_csv('../input/robot-kinematics-dataset/robot_kinematics.csv')`

The notebook includes several exploratory visualizations, such as: Line plots to observe trends in kinematic variables
Scatter plots to examine relationships between features.

The preprocessing steps are relatively minimal and focus on basic data inspection: The dataset is checked for missing values. No advanced imputation techniques are applied. No complex feature engineering or structured preprocessing pipelines are implemented.

Source: EdmilsonRodrigues. (n.d.). GitHub - EdmilsonRodrigues/machine_failure_prediction: This is a machine learning project on top of the base <https://www.kaggle.com/datasets/umerrtx/machine-failure-prediction-using-sensor-data/data> in order to foresee maintenance needs. GitHub.

https://github.com/EdmilsonRodrigues/machine_failure_prediction

What it is: An open-source GitHub repo that implements a full machine learning pipeline for predicting machine failures using sensor data. How they load the dataset: The repository includes download and load scripts (get_data.py) that fetch the raw data from a Kaggle source (Machine Failure Prediction dataset) and place it into the data/ folder.

Visualizations used: While primarily script-based, the project outputs confusion matrices, ROC curves, and model metrics (classification reports) that serve as visual evaluation.

Preprocessing implemented: Feature selection & transformation Categorical conversion Log transformations to normalize skewed features Train/test splitting with stratification Standard ML model comparison (Random Forest, Logistic Regression, Naive Bayes)

✓ Dataset Loading & Basic Exploration

```
#install and set up kaggle
```

```
!pip -q install kaggle
```

```
#Upload kaggle.json  
from google.colab import files  
files.upload()
```

Choose Files  kaggle.json

kaggle.json(application/json) - 63 bytes, last modified: n/a - 100% done

Saving kaggle.json to kaggle.json

```
{'kaggle.json': b'{"username": "hanamuh", "key": "2ecbf649f0e281e4df9d6d84a0d1a58e"}'}
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!ls -la ~/.kaggle
```

```
total 16
drwxr-xr-x 2 root root 4096 Feb 24 00:08 .
drwx----- 1 root root 4096 Feb 24 00:08 ..
-rw----- 1 root root   63 Feb 24 00:10 kaggle.json
```

```
#Download and unzip dataset
```

```
!mkdir -p /content/data
!kaggle datasets download -d ziya07/robotic-operations-performance-dataset -p /content/data --un
!ls -lh /content/data
```

Dataset URL: <https://www.kaggle.com/datasets/ziya07/robotic-operations-performance-dataset>

License(s): CC0-1.0

Downloading robotic-operations-performance-dataset.zip to /content/data

0% 0.00/7.91k [00:00<?, ?B/s]

100% 7.91k/7.91k [00:00<00:00, 29.1MB/s]

total 44K

```
-rw-r--r-- 1 root root 41K Feb 24 00:10 robot_dataset.csv
```

```
#load dataset from CSV
```

```
import pandas as pd
```

```
import numpy as np
```

```
# The file was unzipped as 'robot_dataset.csv'
```

```
df = pd.read_csv('robot_dataset.csv')
```

```
display(df.head())
```

```
print(df.shape)
```

```
df.info()
```

```
display(df.describe())
```

	Robot_ID	Task_Type	Component_ID	Sensor_Type	Sensor_Data	Processing_Time (s)	Accuracy (%)	Environment
0	RBT_001	Inspection	CMP_460	LIDAR	1 (obstacle detected)	67.0	90.4	
1	RBT_002	Assembly	CMP_252	Thermal	85.3 (°C)	71.2	98.1	
2	RBT_003	Inspection	CMP_248	Thermal	92% (visual fit)	49.2	95.3	
3	RBT_004	Welding	CMP_433	Camera	98% (defect-free)	74.5	90.2	
4	RBT_005	Assembly	CMP_992	Camera	92% (visual fit)	64.5	97.2	

(500, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Robot_ID	500 non-null	object
1	Task_Type	500 non-null	object
2	Component_ID	500 non-null	object
3	Sensor_Type	500 non-null	object
4	Sensor_Data	500 non-null	object
5	Processing_Time (s)	500 non-null	float64
6	Accuracy (%)	500 non-null	float64
7	Environmental_Status	500 non-null	object
8	Energy_Consumption (kWh)	500 non-null	float64
9	Human_Intervention_Needed	500 non-null	object
10	Obstacle_Detected	500 non-null	object
11	Defect_Detected	500 non-null	object

dtypes: float64(3), object(9)
memory usage: 47.0+ KB

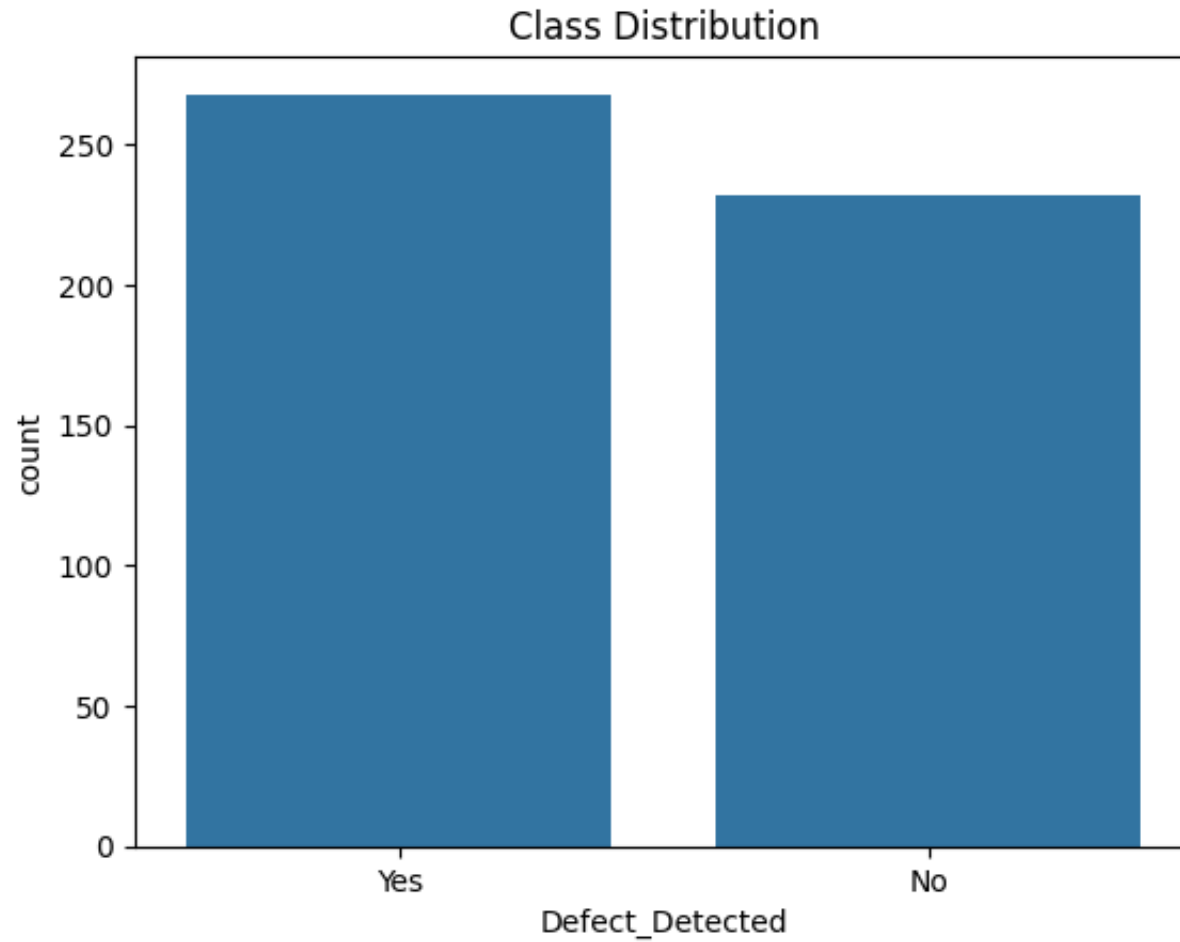
	Processing_Time (s)	Accuracy (%)	Energy_Consumption (kWh)
count	500.000000	500.000000	500.000000

mean	59.937000	94.920600	2.014000
std	11.688808	2.900949	0.582789
min	40.100000	90.000000	1.000000
25%	49.675000	92.500000	1.500000
50%	59.600000	94.900000	2.000000
75%	70.000000	97.500000	2.500000
max	80.000000	99.900000	3.000000

✓ 3. Exploratory Data Analysis (EDA)

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x=df['Defect_Detected'])
plt.title("Class Distribution")
plt.show()
```



```
df['Defect_Detected'].value_counts()  
df['Defect_Detected'].value_counts(normalize=True)*100
```

	proportion
Defect_Detected	
Yes	53.6
No	46.4

dtype: float64

The class distribution is nearly balanced: 53.6% defect (1) and 46.4% non-defect (0). Therefore, class imbalance correction techniques such as SMOTE were not necessary.

```
df.isna().sum()  
(df.isna().mean()*100).sort_values(ascending=False).head(10)
```

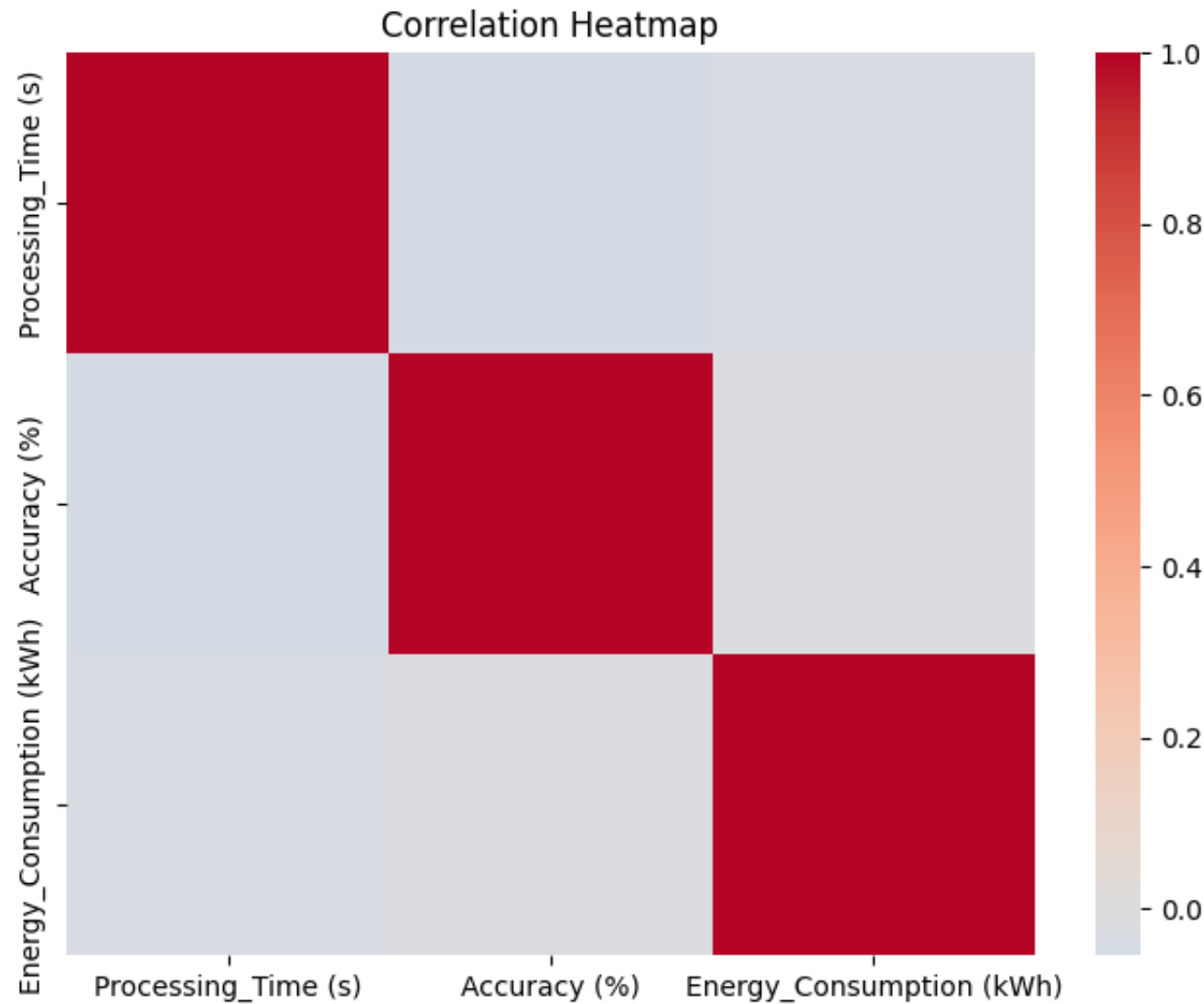
	0
Robot_ID	0.0
Task_Type	0.0
Component_ID	0.0
Sensor_Type	0.0
Sensor_Data	0.0
Processing_Time (s)	0.0
Accuracy (%)	0.0
Environmental_Status	0.0
Energy_Consumption (kWh)	0.0
Human_Intervention_Needed	0.0

dtype: float64

Missing values were checked across all features, and no missing values were found (0%). Therefore, no imputation was required.

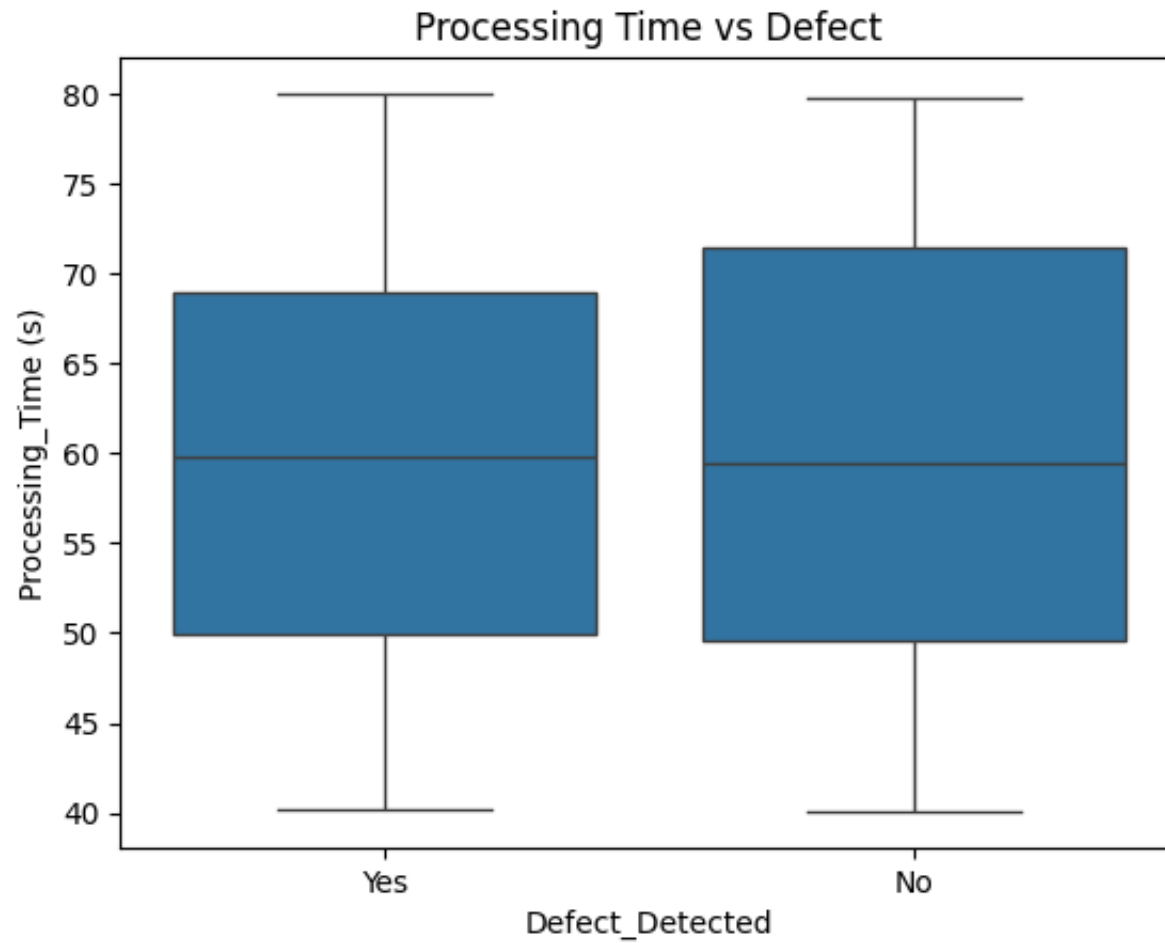
```
numeric_cols = df.select_dtypes(include=['float64','int64'])  
plt.figure(figsize=(8,6))  
sns.heatmap(numeric_cols.corr(), cmap='coolwarm', center=0)  
plt.title("Correlation Heatmap")
```

```
plt.show()
```



Correlation analysis indicates moderate relationships between numerical features with no severe multicollinearity. Therefore, VIF-based feature elimination was not applied.

```
sns.boxplot(x=df['Defect_Detected'], y=df['Processing_Time (s)'])  
plt.title("Processing Time vs Defect")  
plt.show()
```



Boxplot analysis shows overlapping distributions between classes, suggesting limited separability of individual features. This may explain why baseline models achieved performance close to random guessing.

✓ 4.Feature Typing and Preprocessing

```
#Clean column names
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
df.columns
```

```
Index(['robot_id', 'task_type', 'component_id', 'sensor_type', 'sensor_data',
      'processing_time(s)', 'accuracy(%)', 'environmental_status',
      'energy_consumption(kwh)', 'human_intervention_needed',
      'obstacle_detected', 'defect_detected'],
      dtype='object')
```

```
# Convert binary yes/no fields into numeric 0/1
```

```
binary_columns = ["human_intervention_needed", "obstacle_detected", "defect_detected"]
for col in binary_columns:
    if col in df.columns:
        df[col] = df[col].map({"Yes": 1, "No":0})
```

```
#Drop ID columns not useful for predictors
id_columns = ["robot_id", "component_id", "sensor_data"]
for col in id_columns:
    if col in df.columns:
        df.drop(columns=[col], inplace=True)

print("Shape after cleaning:", df.shape)
display(df.head())
```

Shape after cleaning: (500, 9)

	task_type	sensor_type	processing_time_(s)	accuracy_(%)	environmental_status	energy_consumpt
0	Inspection	LIDAR	67.0	90.4	Stable	
1	Assembly	Thermal	71.2	98.1	Stable	
2	Inspection	Thermal	49.2	95.3	Unstable	
3	Welding	Camera	74.5	90.2	Stable	
4	Assembly	Camera	64.5	97.2	Unstable	

We dropped robot_id and component_id as they were just noninformative identifiers. As well as sensor_data because the dataset was messy with different numerical and text inputs.

```
#Basic exploration after cleaning dataset
print("Shape:", df.shape)
df.info()

#Summary stats for numeric columns
df.describe(include="number").T
```

```
#missing value check
df.isna().sum().sort_values(ascending=False)
```

```
Shape: (500, 9)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   task_type                            500 non-null    object
1   sensor_type                          500 non-null    object
2   processing_time_(s)                  500 non-null    float64
3   accuracy_(%)                         500 non-null    float64
4   environmental_status                 500 non-null    object
5   energy_consumption_(kwh)             500 non-null    float64
6   human_intervention_needed            500 non-null    int64
7   obstacle_detected                    500 non-null    int64
8   defect_detected                      500 non-null    int64
dtypes: float64(3), int64(3), object(3)
memory usage: 35.3+ KB
```

	0
task_type	0
sensor_type	0
processing_time_(s)	0
accuracy_(%)	0
environmental_status	0
energy_consumption_(kwh)	0
human_intervention_needed	0
obstacle_detected	0

defect_detected 0

dtype: int64

```
#Defining target before preprocessing
#Define prediction target(label)
target = "human_intervention_needed"
#Split features(X) and label(y) BEFORE preprocessing to avoid leakage
X = df.drop(columns=[target]).copy()
y = df[target].copy()
```

```
#Feature engineering: check for any meaningful ratios
#(accuracy per second, accuracy per kWh, etc)
if "accuracy_%" in X.columns and "processing_time(s)" in X.columns:
    X["efficiency_score"] = X["accuracy_%"] / X["processing_time(s)"]

if "accuracy_%" in X.columns and "energy_consumption_(kwh)" in X.columns:
    X["energy_efficiency"] = X["accuracy_%"] / X["energy_consumption_(kwh)"]

X[["efficiency_score", "energy_efficiency"]].head()

# Ensure column names match the lowercase naming convention
corr_cols = ["accuracy_%", "processing_time(s)", "energy_consumption_(kwh)", "efficiency_score"]
corr = X[[c for c in corr_cols if c in X.columns]].corr()
display(corr)
```

	accuracy_%	processing_time(s)	energy_consumption_(kwh)	efficiency_score
accuracy_%	1.000000	-0.054902	-0.013720	0.204656
processing_time(s)	-0.054902	1.000000	-0.035290	-0.971538
energy_consumption_(kwh)	-0.013720	-0.035290	1.000000	0.037712
efficiency_score	0.204656	-0.971538	0.037712	1.000000
energy_efficiency	0.094711	0.040517	-0.957264	-0.026000

Next steps:

[Generate code with corr](#)[New interactive sheet](#)

We wanted to see other variables that could impact target. This shows how much accuracy we get per second: so robot that is accurate and fast is high score. accurate but slow is lower score, and fast but inaccurate a lower score. So this feature will capture performance tradeoffs. Compressing two variables into one meaningful metric. With the second variable, we test how accurate is robot per unit of energy. High value is accurate and energy efficient and low value is inaccurate and not energy efficient. The results in the table with accuracy vs time is -.05 and vs energy is -.01 so basically 0. Doesn't matter faster or slower robot doesn't impact accuracy and energy doesn't predict accuracy. Efficiency is just high with -.97 and -.96 because as time increases, efficiency goes down. It isn't new information. With Efficiency vs accuracy with values of .20 and .09 this also didn't reflect accuracy.

4.1 Feature typing & Preprocessing

```
#identify numeric vs categorical features
```

```
numeric_features = X.select_dtypes(include=["int64","float"]).columns.tolist()
```

```
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()
```

```
print("Numeric:", numeric_features)
```

```
print("Categorical:", categorical_features)
```

```
Numeric: ['processing_time_(s)', 'accuracy_(%)', 'energy_consumption_(kwh)', 'obstacle_detected',  
Categorical: ['task_type', 'sensor_type', 'environmental_status']
```

Feature, Type, Description

processing_time_(s):Numerical (Continuous), Time required to complete robotic task

accuracy_(%):Numerical (Continuous)Task performance accuracy percentage

energy_consumption_(kwh):Numerical (Continuous), Energy used during task execution

obstacle_detected:Numerical (Binary) Whether obstacle was detected (0 = No, 1 = Yes)

defect_detected:Numerical (Binary),Whether defect was detected (0 = No, 1 = Yes)

task_type:Categorical (Nominal), Type of robotic operation performed

sensor_type:Categorical (Nominal), Type of sensor used

environmental_status:Categorical (Nominal), Stability of operating environment

efficiency_score: Numerical(derived), accuracy divided by processing time.

energy_efficiency: Numerical(derived), accuracy divided by energy consumption

```
#Train/test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, strati
```

✓ 4.2 Preprocessing Steps

#Preprocessesing pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("numeric", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

Features were split into numerical and categorical groups based on data type. Numeric features we used median imputation, it is more robust to outliers and standardized using zscore scaling so all numeric features share same scale. Categorical missing values used 'most frequent', Categorical encoding used 'onehot' encoding scaling for binary indicator columns, in order to not have artificial ordering.

#Before vs AftEr preprocessing

```
print("Raw feature shape:", X_train.shape)
```

```

print("Before preprocessing:")
display(X_train.head())

X_train_preprocessed = preprocessor.fit_transform(X_train)
X_test_preprocessed = preprocessor.transform(X_test)

print("AFTER shapes:", X_train_preprocessed.shape, X_test_preprocessed.shape)

feature_names = preprocessor.get_feature_names_out()
X_train_preprocessed_df = pd.DataFrame(X_train_preprocessed, columns=feature_names)
X_test_preprocessed_df = pd.DataFrame(X_test_preprocessed, columns=feature_names)

print("AFTER preprocessing:")
display(X_train_preprocessed_df.head())

```

Raw feature shape: (400, 10)

Before preprocessing:

	task_type	sensor_type	processing_time_(s)	accuracy_(%)	environmental_status	energy_consumption_(kwh)
161	Inspection	Camera	69.6	94.0	Unstable	
310	Inspection	LIDAR	47.0	98.9	Unstable	
34	Welding	Thermal	66.2	98.3	Unstable	
448	Painting	Thermal	44.6	95.4	Stable	
23	Welding	Camera	55.1	92.0	Stable	

AFTER shapes: (400, 17) (100, 17)

AFTER preprocessing:

	numeric__processing_time_(s)	numeric__accuracy_(%)	numeric__energy_consumption_(kwh)	numeric__environmental_status
0	0.833214	-0.322627	1.319739	Unstable
1	1.100110	1.017005	0.100000	Unstable

1	-1.103446	1.347605	0.129636
2	0.541858	1.143086	0.299651
3	-1.309109	0.154582	1.489754
4	-0.409334	-1.004355	0.299651

After preprocesssing, both training and test sets have 17 features. Categorical variables were expanded via onehot encoding, and numeric features were standardized.

```
#Linear and decision tree
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import numpy as np

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

models = {
    "LogisticRegression": LogisticRegression(max_iter=5000),
    "DecisionTree": DecisionTreeClassifier(random_state=42)}

for name, model in models.items():
    pipe = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("model", model)
    ])

```

```

#cross-val on training set
scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="accuracy")
print(f"\n--- {name} ---")
print("CV accuracy scores:", scores)
print("Mean CV accuracy:", np.mean(scores))
print("Std CV accuracy:", scores.std())

#Fit once and evaluate on held out test set
pipe.fit(X_train, y_train)
preds = pipe.predict(X_test)

print(f"{name} Test Accuracy: {accuracy_score(y_test, preds):.4f}")
print(classification_report(y_test, preds))
print(confusion_matrix(y_test, preds))

```

```

--- LogisticRegression ---
CV accuracy scores: [0.4625 0.425  0.475  0.5375 0.4875]
Mean CV accuracy: 0.4774999999999999
Std CV accuracy: 0.036571847095819475
LogisticRegression Test Accuracy: 0.4900

```

	precision	recall	f1-score	support
0	0.48	0.49	0.48	49
1	0.50	0.49	0.50	51
accuracy			0.49	100
macro avg	0.49	0.49	0.49	100
weighted avg	0.49	0.49	0.49	100

```

[[24 25]
 [26 25]]

--- DecisionTree ---
CV accuracy scores: [0.45  0.4625 0.4375 0.5625 0.4875]

```

```

Mean CV accuracy: 0.48
Std CV accuracy: 0.04444097208657794
DecisionTree Test Accuracy: 0.5000
      precision    recall  f1-score   support

         0         0.49      0.47      0.48         49
         1         0.51      0.53      0.52         51

   accuracy          0.50          0.50          0.50          100
  macro avg          0.50          0.50          0.50          100
weighted avg          0.50          0.50          0.50          100

[[23 26]
 [24 27]]

```

We compared two models, logistic regression as a linear baseline and decision tree as a nonlinear baseline. Logistic regression had a mean CV of .48 and Std of .037. Decision tree had mean CV of .48 and std of .044. so they performed around the same. With the test set of 100 rows. logistic had accuracy of .49 and the decision tree had .50. This split the decision tree was a little bit better.

```
# Model improvement with hyperparameter tuning

from sklearn.model_selection import GridSearchCV

#Search space for logistic regression
param_grid = {
    "model__C": [.01, .1, 1, 10],
    "model__class_weight": ["balanced", None]
}

#Pipeline = preprocessing and model preventing leakage
pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", LogisticRegression(max_iter=5000))
])

#Grid search with 5 fold CV on training data
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="accuracy")
grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)
print("Best CV score:", grid.best_score_)
print("Test accuracy", grid.score(X_test, y_test))
```

```
Best params: {'model__C': 0.01, 'model__class_weight': None}
Best CV score: 0.5
Test accuracy 0.53
```

We attempted to improve performance of logistic regression using GridSearchCV to tune hyperparameters. We searched values of C (regularization strength) where smaller values mean stronger regularization and larger values allow model to fit more flexible patterns. Also tested `class_weight` to determine whether weighting classes improve performance. Best CV was .50 and Test accuracy was .53. The $C=.01$ was chosen as optimal regularization. Meaning model prefers simpler decision boundaries to improve generalization performance, and avoids noisy features

```
#Print final confusion matrix after hyperparameter tuning
from sklearn.metrics import confusion_matrix, classification_report
#Evaluate best model on held out test set
best_model = grid.best_estimator_
preds = best_model.predict(X_test)

print("Best params:", grid.best_params_)
print("Best CV score:", grid.best_score_)
print("Test accuracy:", best_model.score(X_test, y_test))
print(classification_report(y_test, preds))
print("Confusion_matrix:\n", confusion_matrix(y_test, preds))
```

Best params: {'model__C': 0.01, 'model__class_weight': None}

Best CV score: 0.5

Test accuracy: 0.53

	precision	recall	f1-score	support
0	0.52	0.49	0.51	49
1	0.54	0.57	0.55	51
accuracy			0.53	100
macro avg	0.53	0.53	0.53	100
weighted avg	0.53	0.53	0.53	100

Confusion_matrix:

```
[[24 25]
 [22 29]]
```

Hyperparameter tuning improved logistic regression from .50 accuracy to .53 test accuracy. The optimal model used $C=.01$. The confusion matrix shows balanced errors across both classes. With 24 correctly predicted class 0 cases, true negatives and 25 false positives (predicted 1 actually 0), 22 false negative, (predicted class 0 but actually 1) and 29 correctly predicted class 1 cases. Errors are balanced across classes, model isn't biased to either 0 or 1.

✓ 5. Comparison with Papers and Open-source Code

The preprocessing we implemented differed slightly from prior research in many ways. In the study by Yadav et al. SMOTE was applied to address class imbalance and min-max scaling, they did feature selection based on correlation heatmaps, and assessed multicollinearity using VIF before modeling. Our approach did not need SMOTE because the classes were balanced. We used StandardScaler for numerical normalization, and did not use VIF. The other article with S. Pookkuttath, they used normalized vibration signals to a range of -1 to +1 and depended on CNN architecture components such as pooling and dropout for dimensionality reduction and regularization. For our research, we used classic preprocessing techniques such as median imputation, one-hot encoding for categorical variables, and scikit-learn pipelines for reproducibility. In the quantum-based study they embedded the preprocessing into the quantum extraction framework, where ours separated data cleaning, encoding, scaling, and model training using a reproducible pipeline structure. Overall, our project was a more structured classical preprocessing approach focused on reproducibility while the other studies were more on model architecture and performance optimization. Some advantages for our method is that our was focused on transparency and model reproducibility by thoroughly separating data cleaning, encoding, scaling, and model training using a scikit-learn pipeline. Reducing risk of data leakage. A possible disadvantage our preprocessing focuses on classical feature engineering and not automated feature extraction methods like CNN based pooling or deep learning architecture. This may reduce models ability to capture possible nonlinear patterns in high dimensional sensor data. The study from UC Irving, applied minmax

normalization to scale all feature unlike ours which used standardscaler. Min-max scaler preserves the relative distances within a fixed interval, where standardization is often for linear models like logistic regression. The feature selection portion the authors applied many dimensionality reductions techniques, their approach focused on that before modeling. We did not do that, we instead relied on correlation analysis and manual feature engineering to remove less useful features. Their approach was more advanced and research focused and involved more deep learning-based feature extraction

The EdmilsonRodrigues Machine Failure Prediction repository implements a structured, end-to-end machine learning pipeline for sensor-based failure classification, including automated dataset loading, preprocessing, stratified train/test splitting, and multi-model evaluation (e.g., Logistic Regression, Random Forest, and Naive Bayes) with performance metrics, such as confusion matrices and ROC curves. Similar to that repository, our project also addresses a binary classification problem using industrial sensor data and applies proper train/test splitting with stratification. However, our implementation focuses more on explicit data cleaning, feature engineering (e.g., efficiency and risk metrics), and careful prevention of data leakage by separating features and target variables prior to preprocessing. While their repository has a more production-oriented structure, our approach is more transparent and interpretable for predicting human intervention events.

✓ 6. Team Contribution Statement

Athena Lieu: Open-source code documentations, wrote code in Notebook for Section 3 and 4 and also did the comparison and open-source code paragraph

Seonwoo Kim: Added the EDA section, including class distribution analysis, correlation heatmap, and boxplots. Wrote the explanations based on the results and connected them to our preprocessing decisions. Added two research articles to the literature review and summarized their main findings. Also found and summarized one Kaggle open source implementation related to our dataset.

Hana Muhsin: Literature review for a,b,c,f. Collaborated with Athena with Dataset loading and exploration, feature typing and preprocessing. Did Linear and decisiontree comparison and model hyperparameter improvement. Provided explanations and reasonings for sections. Comparison with the literature review for a,b,c,f

